

GUIDE DU FORMATEUR

Pseudo-Code : les concepts clés

À expliquer AVANT le quiz — schémas, patterns & exemples

Variable

Affectation ←

Afficher

MOD

SI / SINON

Boucle

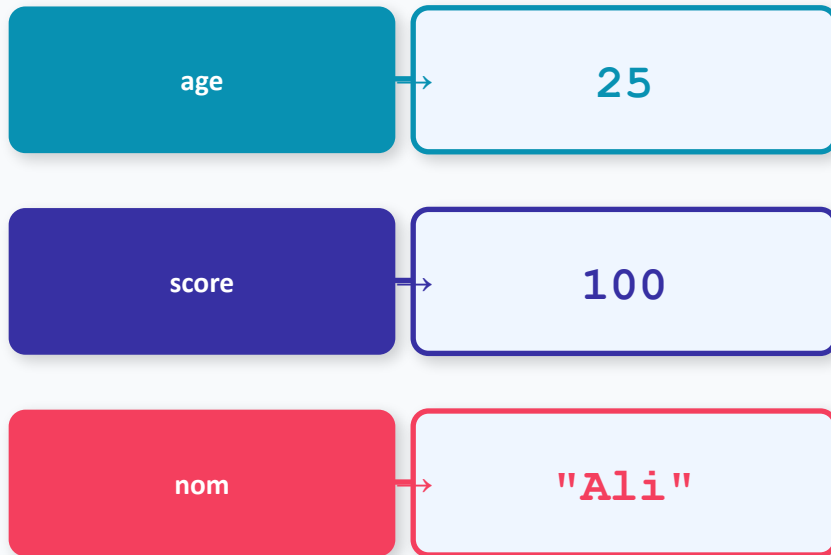
Concaténation



Variable

Définition

Une variable est une boîte nommée qui stocke une valeur en mémoire. Elle peut changer de valeur au cours du programme.



Nom de la variable

Valeur stockée



Règle : une variable garde sa DERNIÈRE valeur assignée — les anciennes valeurs sont perdues.

← Affectation

Syntaxe

variable ← **valeur** / **expression**

"Mettre la valeur de DROITE dans la boîte de GAUCHE"

Trace pas-à-pas :

1. **x** ← 5 → **x = 5**

2. **x** ← **x** + 3 → **x = 8**

3. **y** ← **x** * 2 → **x=8, y=16**

Comment ça marche :



⚠ Le ← n'est PAS une équation mathématique !
x ← x + 1 signifie : x prend sa valeur + 1

```
a ← 10  
b ← a + 5  
afficher b
```

→ 15



Afficher

Texte entre guillemets

Code :

```
afficher "Bonjour"
```

Résultat :

Bonjour

Variable → son contenu

Code :

```
x ← 42  
afficher x
```

Résultat :

42

Ordre = ordre du code

Code :

```
afficher 1  
afficher 2
```

Résultat :

1 2



Règle d'or : chaque instruction afficher = une nouvelle ligne dans la sortie. L'ordre dans le code = l'ordre de l'affichage.

÷ MOD — Le Reste de Division



Analogie : 17 pizzas pour 5 personnes

$$17 \text{ DIV } 5 = 3$$

← quotient (combien de fois on divise)



$$17 \text{ MOD } 5 = 2$$

← reste (ce qui reste)

Parité

```
n MOD 2 = 0  
→ n est pair
```

Dernier chiffre

```
n MOD 10  
→ dernier chiffre
```

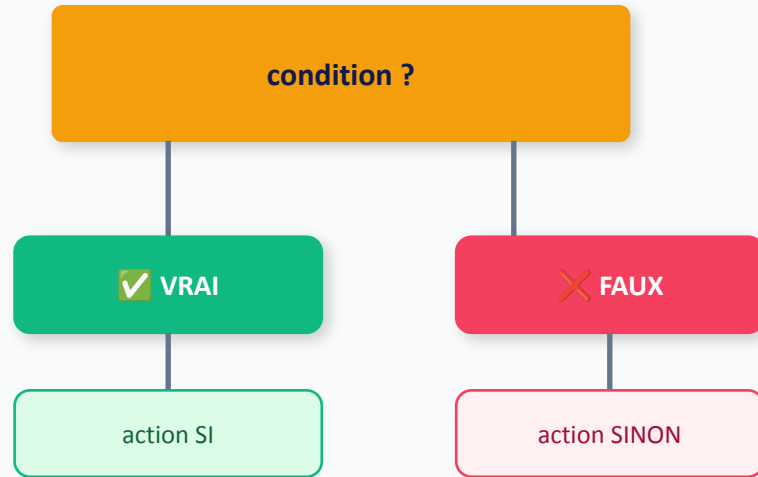
Cyclique

```
i MOD 3  
→ 0, 1, 2, 0, 1...
```



SI / SINON — Conditions

```
si condition alors
  // bloc si vrai
sinon
  // bloc si faux
fin si
```



SI Imbriqué — conditions dans des conditions

```
si note >= 10 alors
  si note >= 16 alors
    afficher "Très bien"
  sinon
    afficher "Bien"
  fin si
sinon
  afficher "Insuffisant"
fin si
```



Indentation = niveau d'imbrication
Chaque SI a son propre fin si
Lire de haut en bas !



Boucle POUR — Répéter N fois

```
pour i de 1 a 5 faire  
    afficher i  
fin pour
```

Visualisation des tours :

i=1

→ 1

i=2

→ 2

i=3

→ 3

i=4

→ 4

i=5

→ 5



POUR : nombre de tours connu à l'avance | TANT QUE : nombre de tours inconnu



Boucle TANT QUE

Quand on ne connaît pas d'avance
le nombre de répétitions :

```
tant que x > 0 faire  
    x ← x - 1  
fin tant que
```



La condition est vérifiée AVANT
chaque tour — si elle est fausse
dès le début, la boucle ne tourne pas.



Boucles Imbriquées

```
pour i de 1 a 3 faire
  pour j de 1 a 2 faire
    afficher i + "-" + j
  fin pour
fin pour
```

Grille de résultats :

i \ j	j=1	j=2
i=1	1-1	1-2
i=2	2-1	2-2
i=3	3-1	3-2

Comment lire les boucles imbriquées :

- 1 Boucle EXTERNE : i démarre à 1
- 2 Boucle INTERNE : j parcourt 1 puis 2
- 3 Pour chaque i, j repart TOUJOURS de 1
- 4 La boucle interne finit → i avance d'1 cran
- 5 Répéter jusqu'à i = 3 inclus

Total tours = tours_externe × tours_interne
= $3 \times 2 = 6$ affichages



Concaténation — Coller des textes

"Bonjour"

+

" "

+

"Ali"

=

"Bonjour Ali"

Texte + Texte

```
prenom <- "Ali"  
nom <- "Ben"  
full <- prenom + " " + nom  
afficher full
```

→ Ali Ben

Texte + Variable

```
score <- 95  
msg <- "Score : " + score  
afficher msg
```

→ Score : 95

Construire pas à pas

```
code <- "ID"  
code <- code + "-"  
code <- code + "42"  
afficher code
```

→ ID-42

 **Important** : Le + entre textes = coller. Un espace doit être explicitement ajouté : " " — Sans espace: "Ali" + "Ben" = "AliBen"



Récapitulatif — Fiche Formateur



Variable

```
x ← valeur
```

Boîte nommée — garde la
DERNIÈRE valeur



Affectation

```
x ← x + 1
```

Droite calculée d'abord,
résultat mis à gauche



Afficher

```
afficher x  
afficher "Txt"
```

1 instruction = 1 ligne ;
ordre = ordre code



MOD

```
23 MOD 5 → 3
```

Reste de division ; parité :
MOD 2



SI / SINON

```
si cond alors...sinon...fin si
```

Condition évaluée une
seule fois ; chaque SI →
fin si



Boucle POUR

```
pour i de 1 a n faire
```

Nombre de tours connu ;
compteur auto



Concaténation

```
"A" + "-" + "B" → "A-B"
```

Colle les textes ; espace
explicite requis